PROJETO FULLSTACK · GRUPO G10

Divisão do trabalho para 4 integrantes

Sistema de Gestão Acadêmica — Notas e Frequência.
Responsabilidades, arquivos, testes e apresentação cronometrada.

> Objetivo do grupo: entregar uma SPA interna com API REST Java/Spring Boot, PostgreSQL 16, PgBouncer, Docker Compose e mecanismos avançados de segurança, auditabilidade e programação SQL.

Distribuição geral

PESSOA	ÁREA	ENTREGA PRINCIPAL
1	Frontend / telas	Login, painel, CRUD e integração visual com a API
2	Java / Spring Boot	Controllers REST, segurança e JDBC parametrizado
3	PostgreSQL / SQL	Modelagem, function, procedure, triggers, views
4	Segurança / Docker	Roles, RLS, rede, timeouts, PgBouncer e testes

Tecnologias e escopo

- Backend: Java 21 + Spring Boot 3.5.16 — obrigatório no enunciado.
- Frontend: JavaScript Vanilla neste ZIP — SPA permitida (React/Next.js opcionais).
- Banco: PostgreSQL 16, schemas app/audit, RBAC, RLS e objetos procedurais.
- Inicialização: docker compose up --build — 4 serviços, sendo PgBouncer o quarto.

Observação: os nomes podem ser escritos nas páginas seguintes. Todos devem conhecer o funcionamento geral e testar sua parte integrada ao sistema.

INTEGRANTE 1 · RESPONSABILIDADE INDIVIDUAL

Frontend e experiência de uso

Nome: _____

O que esta pessoa deve fazer

- Arquivos: frontend/index.html, frontend/app.css, frontend/app.js, frontend/nginx.conf.
- Revisar navegação, login, dashboard, tabelas, filtros de perfil e mensagens de validação.
- Conferir o cadastro, a edição e a exclusão de alunos, disciplinas, turmas e matrículas.
- Garantir que lançamento de notas e frequência apareçam corretamente ao consultar a API.
- Verificar funcionamento no navegador e em tela de celular (responsividade).

Como comprovar que concluiu

- Abrir http://localhost:8080 e fazer login como coordenador.
- Cadastrar um aluno, alterar o nome e conferir a listagem; testar sair e entrar novamente.
- Fazer login como Ana e confirmar que não há botões administrativos.

> Fala na banca: Explicação: 0:00–2:00. Demonstração do CRUD: 10:00–11:00.

Entrega individual

Cada integrante deve conseguir indicar os arquivos que mantém, demonstrar sua funcionalidade e explicar como sua parte se integra ao PostgreSQL.

Checklist: □ Arquivos revisados □ Funcionou no Docker □ Teste demonstrado □ Fala ensaiada

INTEGRANTE 2 · RESPONSABILIDADE INDIVIDUAL

Backend Java + Spring Boot

Nome: _____

O que esta pessoa deve fazer

- Arquivos: backend/pom.xml, api/AcademicoController.java, api/AuthController.java, config/DataSources.java, config/DatabaseExecutor.java e security/.
- Explicar controllers REST: GET, POST, PUT, DELETE e tratamento de erros.
- Testar login JWT e restrições de autorização por perfil com @PreAuthorize.
- Garantir queries parametrizadas pelo JdbcTemplate (prepared statements).
- Comprovar que o Java chama sp_lancar_notas e executa SQL com identidade de usuário em transação.

Como comprovar que concluiu

- Entrar com usuário correto e usuário incorreto; conferir resultados.
- Editar notas na SPA e explicar o caminho HTTP → controller → JDBC → PgBouncer → PostgreSQL.
- Identificar os pools de autenticação, professor e coordenação no DataSources.java.

> Fala na banca: Explicação: 2:00–4:30. Demonstração da procedure: 11:00–12:00.

Entrega individual

Cada integrante deve conseguir indicar os arquivos que mantém, demonstrar sua funcionalidade e explicar como sua parte se integra ao PostgreSQL.

Checklist: □ Arquivos revisados □ Funcionou no Docker □ Teste demonstrado □ Fala ensaiada

INTEGRANTE 3 · RESPONSABILIDADE INDIVIDUAL

PostgreSQL, modelagem e lógica SQL

Nome: _____

O que esta pessoa deve fazer

- Arquivos: bd/backup.sql, docs/modelagem.md, docs/sql-demonstracao.sql.
- Revisar entidades, PKs, FKs, constraints e dados de demonstração (seeds).
- Explicar a function app.fn_situacao_aluno (média, frequência e aprovação).
- Explicar a procedure app.sp_lancar_notas (inserção/atualização de notas).
- Explicar triggers de auditoria e as views vw_boletim e vw_frequencia_resumo.

Como comprovar que concluiu

- Alterar nota e conferir média recalculada na tela.
- Abrir Auditoria e conferir registro UPDATE com dados_antes e dados_depois.
- Consultar as duas views e mostrar resultados reais do banco.

> Fala na banca: Explicação: 4:30–7:30. Demonstração dos triggers/views: 12:00–13:00.

Entrega individual

Cada integrante deve conseguir indicar os arquivos que mantém, demonstrar sua funcionalidade e explicar como sua parte se integra ao PostgreSQL.

Checklist: □ Arquivos revisados □ Funcionou no Docker □ Teste demonstrado □ Fala ensaiada

INTEGRANTE 4 · RESPONSABILIDADE INDIVIDUAL

Segurança PostgreSQL e Docker

Nome: _____

O que esta pessoa deve fazer

- Arquivos: docker-compose.yml, pgbouncer/, bd/pg_hba.conf, bd/postgresql.conf, roles/RLS em backup.sql, scripts/.
- Configurar serviços, volume não versionado, rede interna e PgBouncer.
- Explicar roles rl_leitor, rl_escritor, rl_professor e rl_coordenador; GRANT e RLS.
- Demonstrar limite de IP no pg_hba.conf, SCRAM-SHA-256, statement_timeout e idle timeout.
- Verificar scripts de auditoria do banco, RLS e pools sem depender apenas do código-fonte.

Como comprovar que concluiu

- docker compose ps deve listar 4 serviços.
- testar-rls.ps1 deve mostrar Ana = 3, Bruno = 2 e coordenação = 5 matrículas, além de negar escrita cruzada.
- consultar-pool.ps1 mostra SHOW POOLS; teste do timeout cancela pg_sleep(5).

> Fala na banca: Explicação: 7:30–10:00. Demonstração RLS/pool/timeout: 13:00–15:00.

Entrega individual

Cada integrante deve conseguir indicar os arquivos que mantém, demonstrar sua funcionalidade e explicar como sua parte se integra ao PostgreSQL.

Checklist: □ Arquivos revisados □ Funcionou no Docker □ Teste demonstrado □ Fala ensaiada

PLANO DO GRUPO · APRESENTAÇÃO

15 minutos exatos

O professor exige 10 minutos de arquitetura/configuração e 5 minutos de demonstração prática.

TEMPO	RESPONSÁVEL	ASSUNTO
0:00–2:00	Integrante 1	Produto, telas e SPA
2:00–4:30	Integrante 2	API Java, autenticação e JDBC
4:30–7:30	Integrante 3	Function, procedure, trigger, views e estrutura do banco
7:30–10:00	Integrante 4	Docker, RBAC/RLS, PgBouncer, pg_hba, timeouts
10:00–11:00	Integrante 1	CRUD de aluno no navegador
11:00–12:00	Integrante 2	Lançamento de nota via procedure
12:00–13:00	Integrante 3	Auditoria e views SQL
13:00–15:00	Integrante 4	RLS, SHOW POOLS, teste de timeout e conclusão

Validação conjunta antes de entregar

- □ Rodar o sistema com docker compose up --build e abrir http://localhost:8080.
- □ Demonstrar pelo menos um CRUD completo (aluno ou disciplina).
- □ Executar scripts: verificar-requisitos.ps1, testar-rls.ps1, consultar-pool.ps1.
- □ Conferir function, procedure, 2 views, triggers de auditoria, RLS e roles.
- □ Demonstrar cancelamento pelo statement_timeout e logs de auditoria.
- □ Publicar repositório público no GitHub, sem dados físicos do PostgreSQL.
- □ Ensaiar com cronômetro e deixar terminal/navegador preparados.

> Observação de validação: scripts e código foram revisados estaticamente. Antes de afirmar à banca que o sistema está funcionando, os quatro integrantes precisam executar os testes no Docker do próprio computador.